

AI PM OS Local Shell 完整使用手册

目录

1. 当前完成状态与适用边界
2. 系统组成和事实权威
3. 使用前准备
4. 从 Shell 仓库创建全新项目
5. 新项目第一次初始化
6. 安装和调用独立 ai-pm-os Skill
7. 日常项目管理
8. Pending Updates 与审批
9. 材料、会议和 Decision
10. To-do、Briefing 和报告
11. 敏捷项目管理
12. Dashboard 与数据同步
13. 项目接管和 PM Audit
14. Git 使用和模板升级
15. 推荐工作节奏与提示词
16. 故障排除与已知限制

1. 当前完成状态与适用边界

1.1 当前可用状态

AI PM OS Local Shell 的 M1/P0 已完成，31 项 P0 需求均已验收完成。当前产品基线以本仓库 **main** 分支最新已发布提交为准；运行 **git rev-parse HEAD** 可查看本地克隆对应的精确提交。

当前可用于：

- 新项目初始化和 PM 文件建立
- 需求、范围、WBS、审批和变更治理
- 材料处理和 Pending Updates
- 会议纪要、Action、Decision 和会议索引
- 每日 To-do、Briefing、日报、周报、月报和管理层报告
- Product Backlog、Sprint Backlog、User Story、AC、Story Point、DoR、DoD
- Markdown 到 JSON 主动同步和一致性审计
- React/Vite Dashboard 本地展示
- Git checkpoint、污染检查和发布验证
- 基础项目接管和基础 PM Audit

1.2 尚未完成的长期范围

以下属于后续 P1/P2，不是当前 M1：

- 完整项目接管分析
- 深度跨文件 PM Audit
- Dashboard 高级视觉优化和趋势分析
- Watchdog、后台监听、云服务、多人协作和第三方 PM 工具集成
- macOS 真实设备验证

代码和路径已按跨平台方式设计，但当前只完成 Windows 实机验证。

2. 系统组成和事实权威

2.1 项目壳

```
PROJECT_NAME/  
├─ _AI_GLOBAL_MEMORY/ # Agent 全局工作规则 and 用户偏好  
├─ 00_PM_MEMORY/ # 当前项目运行记忆  
├─ 01_PM_DOCUMENTS/ # 正式项目管理文件  
├─ 02_AGILE/ # 敏捷文件  
├─ 03_MEETINGS/ # 会议文件  
├─ 04_TODO/ # 每日 To-do  
├─ 05_REPORTS/ # 日报、周报、月报、管理层报告  
├─ 06_DASHBOARD/ # React/Vite Dashboard  
├─ 07_DATA/ # JSON 展示和同步数据  
├─ 08_INTAKE/ # 输入材料  
├─ 09_ARCHIVE/ # 归档  
├─ ai-pm-os/ # 内置 Skill  
├─ scripts/ # 仓库级验证和同步脚本  
├─ AGENTS.md # Agent 启动顺序  
└─ USER_GUIDE.md # 本手册
```

2.2 权威关系

必须始终遵守：

1. Approved Markdown Baseline 是最高事实权威。
2. 正式 Markdown 项目文件是日常事实源。
3. 已批准 Pending Update 可以应用到正式文件。
4. Active Context 只用于会话恢复，不能覆盖正式事实。
5. JSON 只用于同步、检查和 Dashboard 展示。
6. Git 提供历史证据，但不能代替业务批准。

禁止从 JSON 反向覆盖 Approved Markdown。

2.3 项目实例边界

从 Shell 仓库 clone 得到的是可初始化的项目模板。新项目只维护自己的项目事实、审批、报告、敏捷数据和 Git 历史，不需要了解或承载 AI PM OS 产品的开发过程记录。

3. 使用前准备

3.1 必需软件

- Git
- Node.js LTS
- npm
- Cursor 或 Codex
- Chrome 或 Microsoft Edge

3.2 检查环境

```
git --version  
node --version  
npm --version
```

3.3 验证 Shell

在项目根目录运行：

```
node scripts/verify-release.js --strict  
node scripts/verify-governance.js  
node ai-pm-os/scripts/validate-skill.js  
node scripts/validate-data.js  
node scripts/check-pollution.js
```

所有命令必须退出 0。

4. 从 Shell 仓库创建全新项目

Shell 仓库地址：

```
https://github.com/Shak-Zhu/AI_PM_OS_LOCAL_SHELL.git
```

该仓库当前为 private 时，clone 电脑必须登录有权限的 GitHub 账号。

4.1 clone 时直接指定新项目名

把最后一个参数写成新项目文件夹名：

```
git clone https://github.com/Shak-Zhu/AI_PM_OS_LOCAL_SHELL.git My_New_Project  
cd My_New_Project
```

`My_New_Project` 可以替换为 `CRM_Migration`、`New_Product_Launch`、`AI_Agent_Project` 等实际名称。

4.2 推荐：建立独立 Git 历史

真实项目不应继续把 Shell 仓库当作 `origin`。

Windows PowerShell:

```
Set-Location My_New_Project
Remove-Item -Recurse -Force .git
git init
git add .
git commit -m "chore: initialize project from AI PM OS shell"
```

macOS/Linux:

```
cd My_New_Project
rm -rf .git
git init
git add .
git commit -m "chore: initialize project from AI PM OS shell"
```

连接新项目仓库:

```
git remote add origin <NEW_PROJECT_GIT_URL>
git branch -M main
git push -u origin main
```

确认 **<NEW_PROJECT_GIT_URL>** 是新项目仓库, 而不是 Shell 模板仓库。

4.3 可选: 保留模板升级来源

如需保留 Shell 仓库作为模板上游:

```
git remote rename origin upstream
git remote add origin <NEW_PROJECT_GIT_URL>
git push -u origin main
git remote -v
```

预期:

```
origin <NEW_PROJECT_GIT_URL>
upstream https://github.com/Shak-Zhu/AI_PM_OS_LOCAL_SHELL.git
```

不要把真实项目内容 push 回 Shell 模板仓库。

4.4 确认是干净壳

```
git status --short
```

预期无输出。

```
node scripts/check-pollution.js
```

预期输出 **RESULT: PASS - Product shell is clean.**

5. 新项目第一次初始化

5.1 打开正确目录

必须使用 Cursor 或 Codex 打开整个项目根目录, 不要只打开 **ai-pm-os/** 或 **06_DASHBOARD/**。

5.2 第一次提示词

请使用 ai-pm-os 管理这个项目。
先完整读取 AGENTS.md，并按启动顺序执行 Memory Boot。
这是一个尚未初始化的新项目，请进入 INIT 工作流。
先询问我最少必要的项目目标、范围、角色、交付日期和验收信息。
在我确认前，所有正式项目文件保持 Draft。
任何未确认事实进入 Gap 或 PM_PENDING_UPDATES.md，不得写成 Approved。
完成后更新 Markdown、JSON 和 Dashboard 数据，并报告写入文件清单。

5.3 需要准备的信息

- 项目名称、目标和业务背景
- 成功标准
- P0/P1/P2 范围和明确排除项
- PM Owner、Project Owner、Sponsor Approver
- Product、Tech、Business、Agile、UAT Owner
- 目标日期或 TBD
- 已知风险和依赖
- Scrum、Kanban、Hybrid 或其他模式

不知道的信息填写 TBD 或 Gap，不要编造。

5.4 初始化后的检查

重点检查：

- 00_PM_MEMORY/PM_CURRENT_STATUS.md
- 00_PM_MEMORY/PM_ROLE_CONFIG.md
- 01_PM_DOCUMENTS/PM_PROJECT_BRIEF.md
- 01_PM_DOCUMENTS/PM_REQUIREMENTS_REGISTER.md
- 01_PM_DOCUMENTS/PM_SCOPE_BASELINE.md
- 00_PM_MEMORY/PM_PENDING_UPDATES.md
- 07_DATA/project_state.json
- 07_DATA/project_roles.json

Scope Baseline 未明确批准前，不应生成正式 WBS。

6. 安装和调用独立 ai-pm-os Skill

6.1 使用完整 Shell

完整 Shell 已包含 ai-pm-os/，通常无需另外安装。

使用当前项目中的 ai-pm-os/SKILL.md。
读取 AGENTS.md 规定的 Memory Boot 文件，并按 ai-pm-os 的路由、事实层级、审批、Pending Updates、Git 和 Pre-send Compliance Gate 执行。

6.2 独立 Skill 仓库结构

独立仓库使用以下结构：

```
AI_PM_OS_SKILL/  
├── ai-pm-os/  
├── SKILL.md  
├── PACKAGE_MANIFEST.md  
├── references/  
├── scenarios/  
└── scripts/
```

安装 URL 应明确指向仓库中的 **ai-pm-os/** 子目录。

6.3 给 Agent 的自动安装指令

独立 Skill 仓库：

```
https://github.com/Shak-Zhu/AI_PM_OS_SKILL
```

Skill 子目录 URL：

```
https://github.com/Shak-Zhu/AI_PM_OS_SKILL/tree/main/ai-pm-os
```

推荐指令：

```
请从以下 Git 仓库安装 ai-pm-os Skill：  
https://github.com/Shak-Zhu/AI_PM_OS_SKILL/tree/main/ai-pm-os
```

要求：

1. Skill 位于仓库的 ai-pm-os/ 子目录，该目录包含 SKILL.md。
2. 安装名称固定为 ai-pm-os。
3. 不要只复制 SKILL.md；必须完整安装 references、scenarios、scripts 和 PACKAGE_MANIFEST.md。
4. 安装后运行 node scripts/validate-skill.js。
5. 验证退出码必须为 0，并确认输出包含 ISOLATED 模式。
6. 告诉我实际安装路径，并提醒我重启 Agent。

6.4 Codex 手动安装

Windows：

```
git clone https://github.com/Shak-Zhu/AI_PM_OS_SKILL.git "$env:TEMP\ai-pm-os-skill-source"  
Copy-Item -Recurse "$env:TEMP\ai-pm-os-skill-source\ai-pm-os" "$env:USERPROFILE\.codex\skills\ai-pm-os"  
node "$env:USERPROFILE\.codex\skills\ai-pm-os\scripts\validate-skill.js"
```

macOS/Linux：

```
git clone https://github.com/Shak-Zhu/AI_PM_OS_SKILL.git /tmp/ai-pm-os-skill-source  
cp -R /tmp/ai-pm-os-skill-source/ai-pm-os ~/.codex/skills/ai-pm-os  
node ~/.codex/skills/ai-pm-os/scripts/validate-skill.js
```

安装后重启 Codex。目标目录已存在时不要直接覆盖。

6.5 Cursor 项目级安装

在目标项目根目录：

Windows PowerShell：

```
git clone https://github.com/Shak-Zhu/AI_PM_OS_SKILL.git "$env:USERPROFILE\.cursor\ai-pm-os-skill-source"
Copy-Item -Recurse "$env:USERPROFILE\.cursor\ai-pm-os-skill-source\ai-pm-os"
"$env:USERPROFILE\.cursor\skills\ai-pm-os"
node "$env:USERPROFILE\.cursor\skills\ai-pm-os\scripts\validate-skill.js"
```

macOS / Linux Bash:

```
git clone https://github.com/Shak-Zhu/AI_PM_OS_SKILL.git ~/.cursor/ai-pm-os-skill-source
cp -R ~/.cursor/ai-pm-os-skill-source/ai-pm-os ~/.cursor/skills/ai-pm-os
node ~/.cursor/skills/ai-pm-os/scripts/validate-skill.js
```

如 Cursor 未自动发现，明确引用：

```
.cursor/skills/ai-pm-os/SKILL.md
```

Cursor 不支持原生 `/ai-pm-os` 时，使用自然语言：

```
请使用 ai-pm-os Skill 执行今日 briefing。
```

6.6 独立包验证

在独立 Skill 仓库根目录：

```
node ai-pm-os/scripts/validate-skill.js
```

预期：

- **Mode: ISOLATED**
- **RESULT: PASS**
- 退出码 0

7. 日常项目管理

每次新会话建议先执行：

```
请使用 ai-pm-os 执行 Memory Boot。
读取正式项目文件恢复状态，告诉我当前阶段、Scope Baseline、正在进行的工作、
待审批项、主要风险和下一安全步骤。不要依赖上一轮聊天记忆。
```

常用意图：

目的	示例
今日状态	使用 ai-pm-os 生成今日 briefing
今日计划	根据Action、风险和Sprint生成今日To-do
处理材料	处理08_INTAKE中的新材料，先生成Pending Updates
会议纪要	处理transcript，生成纪要、Action和Decision摘要
需求更新	登记为Proposed，不要直接改Approved Scope
范围检查	检查Sprint Backlog是否与Scope Baseline冲突
周报	生成周报Markdown、HTML和HTML PPT
Dashboard	刷新Dashboard数据并运行smoke test
审计	执行基础PM Audit，只输出可证实的问题
接管	执行基础Takeover Assessment

8. Pending Updates 与审批

正确流程：

输入材料
→ 事实提取
→ Gap / Conflict 检查
→ Proposed Pending Update
→ Project Owner 审批
→ 原子应用
→ Markdown 更新
→ JSON 同步
→ Dashboard 刷新

批准：

批准 PU-###。应用前先做preflight，必须原子应用；
任何目标文件不能安全写入时，整个PU不应用。

拒绝：

拒绝 PU-###，原因是：.....
保留拒绝记录，不要修改正式项目文件。

要求修改：

PU-### 暂不批准。请按以下要求修改并生成新的独立PU重新审批：.....

禁止静默部分应用、绕过审批或自动执行 **git stash/commit/push**。

9. 材料、会议和 Decision

9.1 材料输入

将材料放入：

08_INTAKE/2026-06-27_customer_requirements/

请使用 ai-pm-os 处理 08_INTAKE/<目录>。
登记Input Log，区分已确认事实、候选事实、冲突、缺失信息和不可读内容。
先生成Pending Updates，不得直接修改Approved Baseline。

不可读材料必须登记 **received-but-unreadable**，不得生成虚构摘要。

9.2 会议处理

请处理以下meeting transcript：
<内容或文件路径>

输出专业会议纪要、Action清单、Decision摘要、Gap和Pending Updates。

会议中讨论过的方案不等于正式 Decision。只有明确确认内容才能进入 Decision Log。

Action 至少包含：

- Action ID

- 描述
- Owner
- Due date
- Status
- Next step
- Source meeting

10. To-do、Briefing 和报告

10.1 今日 To-do

根据当前里程碑、逾期Action、风险、审批、Sprint和昨日未完成事项，生成今日To-do。按优先级排序，不得编造截止日期。

10.2 Daily Briefing

生成今日briefing，包含项目RAG、当前阶段、范围批准状态、今日3~5项行动、逾期、阻塞、待审批、Sprint/Backlog摘要和建议会议。

10.3 报告格式

- 日报：Markdown + HTML
- 周报：Markdown + HTML + HTML PPT
- 月报：Markdown + HTML + HTML PPT
- Steering：Markdown + HTML + HTML PPT

生成本周周报三种格式。仅使用已记录事实；缺少来源的数据标记Gap，不要编造趋势或完成状态。

11. 敏捷项目管理

支持 Scrum、Kanban 和 Hybrid。

每个 Story 至少应包含：

- Story ID、标题、用户价值
- Acceptance Criteria
- Story Point
- Priority、Status
- Scope 关联
- Sprint 归属

Sprint Planning：

为Sprint N准备Planning：
检查候选Story是否属于Approved Scope，检查AC、SP、DoR和capacity；
未批准范围不得进入committed Sprint Backlog。

Kanban 检查:

```
检查WIP limit、Blocked aging、缺Owner/next step、Carry-over和超期Action。
```

禁止自动把未完成 Story 滚入下一 Sprint。必须记录原因、剩余工作、重新估算、新 Sprint 归属和 Owner 确认。

12. Dashboard 与数据同步

12.1 首次安装

```
cd 06_DASHBOARD
npm install
```

12.2 同步、测试和构建

```
npm run sync:data
npm run smoke
npm run build
```

12.3 启动

```
npm run dev
```

访问:

```
http://localhost:5173
```

Dashboard 应显示 Project Status、Scope、Risks & Issues、Actions、Approvals、Sprints、Backlog、To-do、Reports、Document Health 和 Milestones。

clean shell 显示 Empty State 是正确行为。

12.4 根目录数据命令

```
node scripts/sync-data.js
node scripts/validate-data.js
node scripts/audit-data-consistency.js
```

在应用 Approved PU、修改正式 Markdown、生成会议/To-do/报告或刷新 Dashboard 后执行。

同步失败时不得覆盖原 JSON。

13. 项目接管和 PM Audit

基础项目接管:

```
对当前目录执行基础项目接管评估。
识别已有文件、缺失文件、明显风险、未确认范围和待补信息,
生成PM_TAKEOVER_ASSESSMENT.md草案。
```

基础 PM Audit:

检查Scope Baseline是否批准、是否存在未审批变更、逾期Action、缺Owner/due date/next step以及Markdown与JSON明显不同步。

完整接管分析和深度跨文件 Audit 属于 P1。

14. Git 使用和模板升级

14.1 日常检查

```
git status --short
git diff --check
git diff
```

14.2 提交

```
node scripts/verify-release.js --strict
git add <明确文件>
git commit -m "docs: update project status"
git push
```

不要默认使用 `git add -A`。

禁止未经确认使用：

```
git reset --hard
git clean -fd
```

14.3 Skill 升级

```
git status --short
git pull --ff-only
node scripts/validate-skill.js
```

升级后重启 Agent。

14.4 Shell 模板升级

如使用 `upstream`：

```
git fetch upstream
git log --oneline HEAD..upstream/main
```

不要直接覆盖真实项目的 `00_PM_MEMORY/`、`01_PM_DOCUMENTS/`、`02_AGILE/`、`03_MEETINGS/`、`04_TODO/`、`05_REPORTS/` 和 `07_DATA/`。

15. 推荐工作节奏与提示词

每天

1. Memory Boot
2. Daily Briefing

3. 今日 To-do
4. 检查阻塞和待审批
5. 日报和数据同步

每周

1. Backlog refinement
2. 风险和 Action 审查
3. Sprint/里程碑检查
4. 周报三格式输出
5. Git checkpoint

每月

1. 月报
2. Scope 与变更审计
3. 文档健康检查
4. RAID 老化检查
5. Dashboard 和数据一致性检查

恢复上下文：

使用ai-pm-os执行Memory Boot。只从正式文件恢复事实，输出当前意图、最后完成步骤、下一安全步骤、待写入、待审批和来源文件。

处理混乱输入：

以下信息可能重复、冲突或缺失。区分事实层级，识别冲突和Gap，禁止猜测。正式更新先生成Pending Updates。

16. 故障排除与已知限制

16.1 Skill 没有触发

1. 确认打开项目根目录。
2. 明确引用 **ai-pm-os/SKILL.md**。
3. 运行：

```
node ai-pm-os/scripts/validate-skill.js
```

4. 使用自然语言：**请使用 ai-pm-os Skill 执行.....**

16.2 Dashboard 没有数据

```
node scripts/sync-data.js
node scripts/validate-data.js
cd 06_DASHBOARD
npm run sync:data
npm run smoke
```

16.3 GitHub push 无法连接

```
git remote -v  
git config --global --get http.proxy  
git config --global --get https.proxy
```

当前电脑使用 v2rayN 时已验证：

```
git config --global http.proxy socks5h://127.0.0.1:10808  
git config --global https.proxy socks5h://127.0.0.1:10808
```

代理端口可能变化，不要在其他电脑盲目照搬。

16.4 污染检查

```
node scripts/check-pollution.js
```

不要通过扩大跳过目录隐藏污染。区分 Shell 产品模板、真实项目数据和 Shell 产品开发治理数据。

16.5 已知限制

- macOS 尚未真实设备验证。
- Skill 原生注册方式依赖 Cursor/Codex 当前版本；无法自动发现时使用手动路径或自然语言。
- 不提供 Watchdog、后台监听、自动通知、云端多人协作。
- 不替代 Jira、Linear 或 Azure DevOps 的完整交互工作流。
- 是否把内容发送到云端模型取决于 Cursor/Codex 平台和账号配置，本项目不承诺离线推理。